

```

#define MOTOR_DIRECTION 0 //If the direction is reversed, change 0 to 1
#define PIN_DIRECTION_RIGHT 3
#define PIN_DIRECTION_LEFT 4
#define PIN_MOTOR_PWM_RIGHT 5
#define PIN_MOTOR_PWM_LEFT 6

const float D = 0.1275;
const float r = 0.033;

float vx;
float omega;

struct SpeedPWM {
    int left;
    int right;
};

void setup() {
    pinMode(PIN_DIRECTION_LEFT, OUTPUT);
    pinMode(PIN_MOTOR_PWM_LEFT, OUTPUT);
    pinMode(PIN_DIRECTION_RIGHT, OUTPUT);
    pinMode(PIN_MOTOR_PWM_RIGHT, OUTPUT);
    delay(5000);
}

void loop() {
    // Move forward
    vx = 0.5; // m/s
    omega = 0; // rad/s
    SpeedPWM speedpwm = inverse_kinematic(vx,omega);
    motorRun(speedpwm.left, speedpwm.right); delay(1000);
    motorRun(0,0); delay(1000);

    // Rotate 90° to the right
    vx = 0;
    omega = -4.285*PI/2;;
    speedpwm = inverse_kinematic(vx,omega);
    motorRun(speedpwm.left, speedpwm.right); delay(1000);
    motorRun(0,0); delay(1000);

    // Move forward
    vx = 0.35;
    omega = 0;
    speedpwm = inverse_kinematic(vx,omega);

```

```

motorRun(speedpwm.left, speedpwm.right); delay(1000);
motorRun(0,0); delay(1000);

// Rotate 90° to the right
vx = 0;
omega = -4.285*PI/2;;
speedpwm = inverse_kinematic(vx,omega);
motorRun(speedpwm.left, speedpwm.right); delay(1000);
motorRun(0,0); delay(1000);

// Move forward
vx = 0.5;
omega = 0;
speedpwm = inverse_kinematic(vx,omega);
motorRun(speedpwm.left, speedpwm.right); delay(1000);
motorRun(0,0); delay(1000);

// Rotate 90° to the right
vx = 0;
omega = -4.285*PI/2;;
speedpwm = inverse_kinematic(vx,omega);
motorRun(speedpwm.left, speedpwm.right); delay(1000);
motorRun(0,0); delay(1000);

// Move forward
vx = 0.35;
omega = 0;
speedpwm = inverse_kinematic(vx,omega);
motorRun(speedpwm.left, speedpwm.right); delay(1000);
motorRun(0, 0); delay(1000);

// Rotate 90° to the right
vx = 0;
omega = -4.285*PI/2;;
speedpwm = inverse_kinematic(vx,omega);
motorRun(speedpwm.left, speedpwm.right); delay(1000);

//Stop
motorRun(0, 0);
delay(10000);
}

void motorRun(int speedl, int speedr) {
    int dirL = 0, dirR = 0;

```

```

if (speedl > 0) {
    dirL = 0 ^ MOTOR_DIRECTION;
} else {
    dirL = 1 ^ MOTOR_DIRECTION;
    speedl = -speedl;
}

if (speedr > 0) {
    dirR = 1 ^ MOTOR_DIRECTION;
} else {
    dirR = 0 ^ MOTOR_DIRECTION;
    speedr = -speedr;
}

digitalWrite(PIN_DIRECTION_LEFT, dirL);
digitalWrite(PIN_DIRECTION_RIGHT, dirR);
analogWrite(PIN_MOTOR_PWM_LEFT, speedl);
analogWrite(PIN_MOTOR_PWM_RIGHT, speedr);
}

int sign(float x) {
    return (x > 0) - (x < 0);
}

float saturate(float phi_dot) {
    if (abs(phi_dot) > 30.3) {
        return 30.29 * sign(phi_dot);
    }
    else {
        return phi_dot;
    }
}

SpeedPWM inverse_kinematic(float vx, float omega) {
    // Cinématique inverse (vitesse de rotation des roues en fonction de la vitesse linéaire et
    // angulaire du corps)
    float phi_dot_left;
    float phi_dot_right;
    phi_dot_left = saturate( (2*vx - omega*D) / (2*r) ); // rad/s;
    phi_dot_right = saturate( (2*vx + omega*D) / (2*r) ); // rad/s;

    // Conversion rad/s -> PWM
    // Vitesse de rotation max (supposée) d'une roue = 30.31 rad/s

```

```
SpeedPWM s;  
s.left = map(abs(phi_dot_left), 0, 30.3, 0, 255) * sign(phi_dot_left);  
s.right = map(abs(phi_dot_right), 0, 30.3, 0, 255) * sign(phi_dot_right);  
return s;  
}
```